

Week2 树状数组（Binary Indexed Tree）

Updated 2026-03-09 23:06 GMT+8
Compiled by Hongfei Yan (2025 Spring)

1 位运算（Bit Manipulation）

- 类别：编程技巧
- 说明：位运算是利用二进制位进行操作（如 &、|、^、<<、>> 等）来高效解决问题的方法。它本身不是算法或数据结构，但广泛用于优化算法、状态压缩、哈希、奇偶判断等场景。
- 典型应用：Lowbit、判断是否为 2 的幂、集合表示（状态压缩 DP）等。

E190.颠倒二进制位

bit manipulation, <https://leetcode.cn/problems/reverse-bits/>

颠倒给定的 32 位有符号整数的二进制位。

示例 1:

输入: n = 43261596

输出: 964176192

解释:

整数	二进制
43261596	00000010100101000001111010011100
964176192	00111001011110000010100101000000

示例 2:

输入: n = 2147483644

输出: 1073741822

解释:

整数	二进制
2147483644	0111111111111111111111111111100
1073741822	0011111111111111111111111111110

提示：

- `0 <= n <= 231 - 2`
- `n` 为偶数

进阶: 如果多次调用这个函数, 你将如何优化你的算法?

位运算 (更贴近底层)

```
1 class Solution:
2     def reverseBits(self, n: int) -> int:
3         res = 0
4         for i in range(32):
5             res = (res << 1) | (n & 1)
6             n >>= 1
7         return res
```

优点: 不依赖字符串操作, 纯位运算, 效率更高。展示对位操作的理解。

说明:

- 每次取 `n` 的最低位 (`n & 1`), 放到 `res` 的末尾。
- `res` 左移腾出位置, `n` 右移取出下一位。
- 循环 32 次确保处理全部位。

M1680.连接连续二进制数字

bit manipulation, <https://leetcode.cn/problems/concatenation-of-consecutive-binary-numbers/>

给你一个整数 `n`, 请你将 `1` 到 `n` 的二进制表示连接起来, 并返回连接结果对应的十进制数字对 `109 + 7` 取余的结果。

示例 1:

```
1 输入: n = 1
2 输出: 1
3 解释: 二进制的 "1" 对应着十进制的 1 。
```

示例 2:

```
1 输入: n = 3
2 输出: 27
3 解释: 二进制下, 1, 2 和 3 分别对应 "1", "10" 和 "11" 。
4 将它们依次连接, 我们得到 "11011", 对应着十进制的 27 。
```

示例 3:

```
1  输入: n = 12
2  输出: 505379714
3  解释: 连接结果为 "1101110010111011110001001101010111100" 。
4  对应的十进制数字为 118505380540 。
5  对  $10^9 + 7$  取余后, 结果为 505379714 。
```

提示:

- $1 \leq n \leq 10^5$

这道题要求我们将从 1 到 n 的所有整数的二进制表示连接起来, 并将结果对 $10^9 + 7$ 取余。

算法思路

对于每一个数字 i , 当我们将其二进制表示拼接到目前的结果 ans 后面时, 相当于将当前的 ans 向左移动 i 的二进制长度, 然后再把 i 加上去。

也就是说: $ans = ((ans \ll length) | i) \% MOD$

我们不需要对每个数都去计算它的二进制长度, 因为一个数的二进制长度 $length$ 只有在遇到 2 的幂 (即 1, 2, 4, 8, 16, ...) 时才会增加 1。

判断一个数 i 是否是 2 的幂, 可以使用非常经典的位运算技巧: $i \& (i - 1) == 0$ 。如果条件成立, 说明 i 是 2 的幂, 我们将 $length$ 增加 1。

这样, 我们就可以在一个 $O(n)$ 的循环中以极小的常数完成所有运算。

Python 3 代码

```
1  class Solution:
2      def concatenatedBinary(self, n: int) -> int:
3          MOD = 10**9 + 7
4          ans = 0
5          length = 0
6
7          for i in range(1, n + 1):
8              # 当 i 是 2 的幂时, 其二进制长度增加 1
9              if i & (i - 1) == 0:
10                 length += 1
11
12             # 将当前结果左移 length 位, 并加上当前的数字 i (由于左移后低位全为0, 可以
            使用按位或 | 代替加法 +)
13             ans = ((ans << length) | i) % MOD
14
15         return ans
```

复杂度分析

- **时间复杂度:** $O(n)$, 我们只需要从 1 遍历到 n , 每一步只进行基础的位运算、加法以及取余等常数时间的操作, 对于 $n \leq 10^5$ 的数据量可以在几毫秒内执行完毕。
- **空间复杂度:** $O(1)$, 仅使用了几个变量来记录当前长度、余数等信息, 不需要额外的存储空间。

M1461.检查一个字符串是否包含所有长度为 K 的二进制子串

bit manipulation, <https://leetcode.cn/problems/check-if-a-string-contains-all-binary-codes-of-size-k/>

给你一个二进制字符串 `s` 和一个整数 `k`。如果所有长度为 `k` 的二进制字符串都是 `s` 的子串，请返回 `true`，否则请返回 `false`。

示例 1:

```
1 输入: s = "00110110", k = 2
2 输出: true
3 解释: 长度为 2 的二进制串包括 "00", "01", "10" 和 "11"。它们分别是 s 中下标为 0, 1, 3, 2 开始的长度为 2 的子串。
```

示例 2:

```
1 输入: s = "0110", k = 1
2 输出: true
3 解释: 长度为 1 的二进制串包括 "0" 和 "1", 显然它们都是 s 的子串。
```

示例 3:

```
1 输入: s = "0110", k = 2
2 输出: false
3 解释: 长度为 2 的二进制串 "00" 没有出现在 s 中。
```

提示:

- `1 <= s.length <= 5 * 105`
- `s[i]` 不是 `'0'` 就是 `'1'`
- `1 <= k <= 20`

这道题的核心任务是判断在给定的二进制字符串 `s` 中，是否包含了所有可能出现的长度为 `k` 的二进制子串。

长度为 `k` 的二进制子串共有 2^k 种组合。为了验证是否所有组合都在 `s` 中出现过，最直观且高效的方法是：遍历 `s` 中所有的长度为 `k` 的子串，将其放入集合（Set）中去重，最后判断集合的大小是否等于 2^k 。

在此思路下，我们可以进行一次早停优化（Early Exit）：

- 字符串 `s` 中长度为 `k` 的子串最多有 `len(s) - k + 1` 个。
- 如果哪怕这 `len(s) - k + 1` 个子串各不相同，它的总数依然小于 2^k ，那么必然无法包含所有

的组合，可以直接返回 `False`，避免后续多余的计算。

方法：滚动哈希 / 位运算（进阶思路）

可以不利用切片生成新的字符串，而是将长度为 k 的二进制字符串看做一个十进制整数。比如子串 "101" 可以用 5 表示。使用位运算能够做到 $O(1)$ 地更新哈希值（即滑动窗口右移一位，左移抛弃最高位，末位加上新进来的字符）。

```
1 class Solution:
2     def hasAllCodes(self, s: str, k: int) -> bool:
3         # 同样进行早停优化
4         req = 1 << k
5         if len(s) - k + 1 < req:
6             return False
7
8         # seen 使用 bytearray 充当布尔数组，速度比 list 快，占用空间也很小
9         seen = bytearray(req)
10        seen = [0]*req
11
12        # 初始化前 k-1 个字符组成的数值
13        num = int(s[:k-1], 2) if k > 1 else 0
14        mask = req - 1
15        count = 0
16
17        # 滑动窗口遍历
18        for i in range(k - 1, len(s)):
19            # 移位，按位与去最高位，加上新进来的最后一位字符
20            num = ((num << 1) & mask) | (1 if s[i] == '1' else 0)
21
22            # 如果是第一次见到这个数字组合
23            if not seen[num]:
24                seen[num] = 1
25                count += 1
26            # 当不同组合凑齐 2^k 种，说明包含了所有情况
27            if count == req:
28                return True
29
30        return False
```

复杂度分析（方法二）

- 时间复杂度： $O(N)$ ，位运算滚动更新只需要 $O(1)$ 时间，整体只需线性扫描一遍。
- 空间复杂度： $O(2^k)$ ，布尔数组固定开辟 2^k 个位置的空间。相比集合存字符串极大地节省了内存占用。

T30201: 旅行售货商问题

bitmask dp, <http://cs101.openjudge.cn/practice/30201/>

一个国家有 n 个城市，每两个城市之间都开设有航班，从城市 i 到城市 j 的航班价格为 $cost[i, j]$ ，而且往、返航班的价格相同。

售货商要从一个城市出发，途径每个城市 1 次（且每个城市只能经过 1 次），最终返回出发地，而且他的交通工具只有航班，请求出他旅行的最小开销。

输入

输入的第 1 行是一个正整数 n ($3 \leq n \leq 18$)

然后有 n 行，每行有 n 个正整数，构成一个 $n * n$ 的矩阵，矩阵的第 i 行第 j 列为城市 i 到城市 j 的航班价格。 $1 \leq \text{cost}[i,j] \leq 10^4$

输出

输出数据为一个正整数 m ，表示旅行售货商的最小开销

样例输入

```
1 | 4
2 | 0 4 1 3
3 | 4 0 2 1
4 | 1 2 0 5
5 | 3 1 5 0
```

样例输出

```
1 | 7
```

提示：dp, dfs

来源：2025fall-cs101 yan

除了用状压dp，还可以有三个层次的优化：I/O 与数据结构优化（基础）、逻辑剪枝（中级）以及 Python 特性优化（高级）。

详细解读优化的三个层次

1. I/O 与数据结构优化（基础）

- **sys.stdin.read**: 原代码在循环里用 `input()`，每次都要处理缓冲区。用 `read().split()` 将所有数据一次性读入内存并切分，配合迭代器 `iter()` 和 `next()`，是 Python 刷题处理大量数据的标准做法。
- **列表 vs 字典**: 原代码用 `d = {}` 存图。字典查询需要哈希计算，而列表（数组）是基于内存偏移量的直接访问。在高频访问场景下，列表要快得多。

2. 逻辑剪枝（中级）

- **range(1, size, 2)**: 这是一个非常漂亮的逻辑剪枝。因为我们规定从城市 0 出发，所以任何合法的状态 `mask`，其二进制最低位（第 0 位）必须是 1。这意味着 `mask` 必然是奇数。直接让循环步长为 2，循环次数瞬间减少一半。
- **range(1, n)**: 内层循环寻找下一个城市 `v` 时，不需要考虑 0。因为 TSP 规定中间过程不走重复路，0 是起点，只有在遍历完所有节点后，计算 `ans` 时才考虑回到 0。

3. Python 特性优化（高级 - 提速关键）

- **去处 min() 函数**: 这是 Python 算法题优化的“杀手锏”。`dp[new][k] = min(dp[new][k], val)` 看起来很简洁。但 Python 的函数调用栈开销很大。在千万级别的循环中，这会严重

拖慢速度。改成 `if val < dp[new][k]: dp[new][k] = val` 虽然代码多了两行，但运行速度会有质的飞跃。

- **局部变量缓存:** `curr_dist = dp[mask][u]`。在 Python 中，访问 `dp[mask][u]` 需要两次列表索引操作。把它存为局部变量 `curr_dist`，后续计算只读这个变量，减少了索引查找开销。

运行时间：3867ms

```
1  import sys
2
3  def solve():
4      # 1. 快速 I/O
5      input = sys.stdin.read
6      data = input().split()
7      iterator = iter(data)
8
9      try:
10         n = int(next(iterator))
11     except StopIteration:
12         return
13
14     # 2. 使用二维列表代替字典，并预处理为整数
15     # 直接读取 n*n 个数据构建矩阵
16     dist = []
17     for _ in range(n):
18         row = [int(next(iterator)) for _ in range(n)]
19         dist.append(row)
20
21     # 3. 初始化 DP
22     # 状态上限 1<<n
23     size = 1 << n
24     inf = float("inf")
25     # 这里的 dp[i][j] 表示：状态掩码为 i，当前停留在 j 城市
26     dp = [[inf] * n for _ in range(size)]
27
28     # 起点固定为 0，初始状态掩码为 1 (二进制 ...001)，花费 0
29     dp[1][0] = 0
30
31     # 4. 逻辑优化：只遍历奇数 mask
32     # 因为起点 0 永远在路径中，所以 mask 的第 0 位永远是 1，即 mask 永远是奇数
33     for i in range(1, size, 2):
34         # 优化：如果当前状态 i 下，所有结尾可能都不可达，直接跳过（但在 TSP 稠密图中较少见）
35
36         for j in range(n):
37             # 如果状态 i 下不可能停在 j，跳过
38             # 或者 if not (i >> j) & 1: continue (隐含条件，通常由 inf 判断即可)
39             if dp[i][j] == inf:
40                 continue
41
42             curr_dist = dp[i][j]
43
44             # 5. 内层循环优化
45             # k 从 1 开始，因为中间过程不可能回到起点 0
```

```

46         for k in range(1, n):
47             # 如果 k 已经在状态 i 中, 跳过
48             if (i >> k) & 1:
49                 continue
50
51             new_mask = i | (1 << k)
52             new_cost = curr_dist + dist[j][k]
53
54             # 6. 移除 min() 函数调用, 手动比较更快
55             if new_cost < dp[new_mask][k]:
56                 dp[new_mask][k] = new_cost
57
58         # 7. 计算最终回路
59         ans = inf
60         # 最终状态必然是 (1<<n) - 1, 即全 1
61         final_mask = size - 1
62
63         # 此时停留在 i, 需要从 i 回到 0
64         for i in range(1, n):
65             cost = dp[final_mask][i] + dist[i][0]
66             if cost < ans:
67                 ans = cost
68
69         print(ans)
70
71 if __name__ == '__main__':
72     solve()

```

36行第2个for循环, `for j in range(n):`

这一层循环的意思是: 枚举当前所在的“终点城市”。为了彻底理解, 需要结合 DP 状态定义来看。

1. 核心含义

`dp[i][j]` 的定义是:

- `i`: 经过了哪些城市 (状态/集合)。
- `j`: 目前停留在哪个城市。

第 2 个循环 `for j in range(n)` 就是在遍历这个“目前停留的城市”。

2. 为什么要遍历它? (举例说明)

假设有 3 个城市: 0, 1, 2。

外层循环 `i` 到了状态 `111` (二进制), 表示 `{0, 1, 2}` 这三个城市都去过了。

虽然大家都去过这三个城市, 但“怎么走的”以及“最后停在哪里”是不同的, 这对下一步去哪里至关重要。

- 情况 A: 路径是 $0 \rightarrow 1 \rightarrow 2$ 。
 - 此时 `j = 2` (当前在 2 号城市)。
 - 如果你下一步要去 3 号城市, 路费是 `distance[2][3]`。
- 情况 B: 路径是 $0 \rightarrow 2 \rightarrow 1$ 。
 - 此时 `j = 1` (当前在 1 号城市)。

- 如果你下一步要去 3 号城市，路费是 `distance[1][3]`。

结论：

只知道“去过哪些城市 (`i`)”是不够的，必须知道“现在脚踩在哪个城市 (`j`)”，才能算出“去下一个城市 (`k`)”的距离。

3. 代码逻辑链条

这个循环的作用起到了承上启下的连接作用：

1. 承上（检查合法性）：

```
1 | if dp[i][j] == float("inf"):
2 |     continue
```

不是所有城市都能作为当前状态的终点。比如，如果集合 `i` 里根本没有城市 `j`，或者从起点根本走不到 `j`，这个状态就是无效的，直接跳过。

2. 启下（状态转移）：

```
1 | dp[newi][k] = min(..., dp[i][j] + d[j][k])
```

这里用到了 `j`。我们要从“当前点 `j`”走到“下一个点 `k`”。如果没有这层 `j` 的循环，我们就不知道这笔路费 `d[j][k]` 里的起点是谁。

总结

第 2 个循环就是在问：

“在已经去过集合 `i` 的所有方案中，如果我们最后停在了城市 `0`、或者城市 `1`、.....或者城市 `n-1`，分别会怎么样？”

Q：为什么在openjudge.cn上提交代码，套在函数中的程序，运行更快？

把程序套在函数里面，就不超时了。因为：局部变量（函数内部定义的变量）存储在 局部命名空间（local namespace）中，通过 索引直接访问（LOAD_FAST 指令），速度非常快。

全局变量（模块级别定义的变量）存储在 全局字典（globals dict）中，每次访问都需要 哈希查找（LOAD_GLOBAL 指令），开销更大。@李睿安

运行时间：4135ms

```
1 | def solve():
2 |     n = int(input().strip())
3 |     cost = []
4 |     for _ in range(n):
5 |         row = list(map(int, input().split()))
6 |         cost.append(row)
7 |
8 |     # 如果只有1个城市? 但题目保证 n>=3
9 |     INF = float('inf')
10 |    # dp[mask][i]: mask 是已访问的城市集合, i 是当前所在城市 (0 <= i < n)
11 |    # mask 是一个整数, bit j 为1 表示城市 j 已访问
12 |    total_masks = 1 << n
```

```

13 dp = [[INF] * n for _ in range(total_masks)]
14
15 # 起点设为城市0
16 dp[1][0] = 0 # 只访问了城市0, 当前在0, 花费0
17
18 # 遍历所有状态
19 for mask in range(1, total_masks, 2):
20     for u in range(n):
21         if dp[mask][u] == INF:
22             continue
23         # 尝试从 u 到未访问的城市 v
24         for v in range(n):
25             if mask & (1 << v):
26                 continue # v 已访问, 跳过
27             new_mask = mask | (1 << v)
28             new_cost = dp[mask][u] + cost[u][v]
29             if new_cost < dp[new_mask][v]:
30                 dp[new_mask][v] = new_cost
31
32 # 所有城市都访问完的状态是 (1 << n) - 1
33 full_mask = total_masks - 1
34 ans = INF
35 for i in range(1, n): # 从其他城市回到起点0
36     if dp[full_mask][i] != INF:
37         ans = min(ans, dp[full_mask][i] + cost[i][0])
38
39 print(ans)
40
41 if __name__ == '__main__':
42     solve()

```

运行时间: 6252ms

```

1 n = int(input().strip())
2 cost = []
3 for _ in range(n):
4     row = list(map(int, input().split()))
5     cost.append(row)
6
7 # 如果只有1个城市? 但题目保证 n>=3
8 INF = float('inf')
9 # dp[mask][i]: mask 是已访问的城市集合, i 是当前所在城市 (0 <= i < n)
10 # mask 是一个整数, bit j 为1 表示城市 j 已访问
11 total_masks = 1 << n
12 dp = [[INF] * n for _ in range(total_masks)]
13
14 # 起点设为城市0
15 dp[1][0] = 0 # 只访问了城市0, 当前在0, 花费0
16
17 # 遍历所有状态
18 for mask in range(1, total_masks, 2):
19     for u in range(n):
20         if dp[mask][u] == INF:

```

```

21         continue
22     # 尝试从 u 到未访问的城市 v
23     for v in range(n):
24         if mask & (1 << v):
25             continue # v 已访问, 跳过
26         new_mask = mask | (1 << v)
27         new_cost = dp[mask][u] + cost[u][v]
28         if new_cost < dp[new_mask][v]:
29             dp[new_mask][v] = new_cost
30
31     # 所有城市都访问完的状态是 (1 << n) - 1
32     full_mask = total_masks - 1
33     ans = INF
34     for i in range(1, n): # 从其他城市回到起点0
35         if dp[full_mask][i] != INF:
36             ans = min(ans, dp[full_mask][i] + cost[i][0])
37
38     print(ans)

```

2 前缀和 (Prefix Sum)

- 类别: 技巧 / 预处理方法
- 说明: 前缀和本身不是一种完整的算法或数据结构, 而是一种常用的预处理技巧, 用于快速计算数组区间和。它通常配合数组使用, 通过 $O(n)$ 预处理实现 $O(1)$ 的区间查询。
- 典型应用: 子数组和、滑动窗口、差分数组等。

E303.区域和检索 - 数组不可变

prefix sum, <https://leetcode.cn/problems/range-sum-query-immutable/>

给定一个整数数组 `nums`, 处理以下类型的多个查询:

1. 计算索引 `left` 和 `right` (包含 `left` 和 `right`) 之间的 `nums` 元素的和, 其中 `left <= right`

实现 `NumArray` 类:

- `NumArray(int[] nums)` 使用数组 `nums` 初始化对象
- `int sumRange(int left, int right)` 返回数组 `nums` 中索引 `left` 和 `right` 之间的元素的总和, 包含 `left` 和 `right` 两点 (也就是 `nums[left] + nums[left + 1] + ... + nums[right]`)

示例 1:

```

1  输入:
2  ["NumArray", "sumRange", "sumRange", "sumRange"]
3  [[[-2, 0, 3, -5, 2, -1]], [0, 2], [2, 5], [0, 5]]
4  输出:
5  [null, 1, -1, -3]
6
7  解释:
8  NumArray numArray = new NumArray([-2, 0, 3, -5, 2, -1]);
9  numArray.sumRange(0, 2); // return 1 ((-2) + 0 + 3)
10 numArray.sumRange(2, 5); // return -1 (3 + (-5) + 2 + (-1))
11 numArray.sumRange(0, 5); // return -3 ((-2) + 0 + 3 + (-5) + 2 + (-1))

```

提示:

- `1 <= nums.length <= 10^4`
- `-10^5 <= nums[i] <= 10^5`
- `0 <= left <= right < nums.length`
- 最多调用 `10^4` 次 `sumRange` 方法

如果只是 查询一次 或 很少查询, 可以直接:

```

1  sum(nums[left:right+1])

```

但如果: 查询次数 $\geq n$, 就必须 前缀和。这也是这题为什么叫 **Range Query**。

这题是经典 前缀和 (**Prefix Sum**) 模板题。思路很直接:

一、核心思想

预处理一个 前缀和数组 `pre`:

`pre[i] = nums[0] + nums[1] + ... + nums[i-1]`

也就是:

```

1  pre[0] = 0
2  pre[1] = nums[0]
3  pre[2] = nums[0] + nums[1]
4  ...

```

这样区间 `[left, right]` 的和就是:

`sum(left,right) = pre[right+1] - pre[left]`

原因:

```

1  pre[right+1] = nums[0] + ... + nums[right]
2  pre[left]    = nums[0] + ... + nums[left-1]

```

相减就得到：

```
1 | nums[left] + ... + nums[right]
```

时间复杂度：

- 初始化： $O(n)$
- 每次查询： $O(1)$

非常适合 多次查询区间和。

二、Python实现

```
1 | from typing import List
2 |
3 |
4 | class NumArray:
5 |
6 |     def __init__(self, nums: List[int]):
7 |         n = len(nums)
8 |         self.pre = [0] * (n + 1)
9 |
10 |        for i in range(n):
11 |            self.pre[i + 1] = self.pre[i] + nums[i]
12 |
13 |        def sumRange(self, left: int, right: int) -> int:
14 |            return self.pre[right + 1] - self.pre[left]
15 |
16 | # Your NumArray object will be instantiated and called as such:
17 | # obj = NumArray(nums)
18 | # param_1 = obj.sumRange(left,right)
```

四、为什么 `pre` 长度是 `n+1`

关键是 避免特判 `left=0`。

如果不用 `n+1`，写法会变成：

```
1 | if left == 0:
2 |     return pre[right]
3 | else:
4 |     return pre[right] - pre[left-1]
```

而使用 `n+1`： `pre[0] = 0`

就可以统一写成： `pre[right+1] - pre[left]`

代码更干净。

五、这题的本质

这是 最基础的前缀和模型：

问题类型	技巧
多次区间求和	前缀和
区间修改	差分
动态更新区间和	Fenwick / Segment Tree

这题本质就是：预处理 $O(n)$ ，查询 $O(1)$

常用的前缀和写法，比标准写法更短、清晰、Pythonic。

三行构造前缀和（推荐写法）

核心技巧：

利用 `itertools.accumulate`

```

1  from typing import List
2  import itertools
3
4  class NumArray:
5
6      def __init__(self, nums: List[int]):
7          self.pre = [0] + list(itertools.accumulate(nums))
8
9      def sumRange(self, left: int, right: int) -> int:
10         return self.pre[right+1] - self.pre[left]
```

初始化只需要一行核心代码：

四、最常见的前缀和模板

以后你在做算法题基本都会看到这个模板：

```

1  from itertools import accumulate
2
3  nums = [1,2,3,4,5]
4
5  pre = [0] + list(accumulate(nums))
6
7  # 区间和
8  def query(l,r):
9      return pre[r+1] - pre[l]
```

五、很多人不知道的一个技巧

`accumulate` 还能做 前缀最大值 / 最小值 / 乘积：

前缀最大值

```

1 from itertools import accumulate
2 import operator
3
4 nums = [3,1,5,2,4]
5
6 pre_max = list(accumulate(nums, max))

```

得到：

```

1 [3,3,5,5,5]

```

前缀乘积

```

1 list(accumulate(nums, operator.mul))

```

顺便掌握：M304.二维区域和检索 - 矩阵不可变

prefix sum, <https://leetcode.cn/problems/range-sum-query-2d-immutable/>

给定一个二维矩阵 `matrix`，以下类型的多个请求：

- 计算其子矩形范围内元素的总和，该子矩形的 左上角 为 `(row1, col1)`，右下角 为 `(row2, col2)`。

实现 `NumMatrix` 类：

- `NumMatrix(int[][] matrix)` 给定整数矩阵 `matrix` 进行初始化
- `int sumRegion(int row1, int col1, int row2, int col2)` 返回 左上角 `(row1, col1)`、右下角 `(row2, col2)` 所描述的子矩阵的元素 总和。

示例 1：

3	0	1	4	2
5	6	3	2	1
1	2	0	1	5
4	1	0	1	7
1	0	3	0	5

```

1  输入:
2  ["NumMatrix","sumRegion","sumRegion","sumRegion"]
3  [[[[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],[4,1,0,1,7],[1,0,3,0,5]]],[2,1,4,3],
   [1,1,2,2],[1,2,2,4]]
4  输出:
5  [null, 8, 11, 12]
6
7  解释:
8  NumMatrix numMatrix = new NumMatrix([[3,0,1,4,2],[5,6,3,2,1],[1,2,0,1,5],
   [4,1,0,1,7],[1,0,3,0,5]]);
9  numMatrix.sumRegion(2, 1, 4, 3); // return 8 (红色矩形框的元素总和)
10 numMatrix.sumRegion(1, 1, 2, 2); // return 11 (绿色矩形框的元素总和)
11 numMatrix.sumRegion(1, 2, 2, 4); // return 12 (蓝色矩形框的元素总和)

```

提示:

- `m == matrix.length`
- `n == matrix[i].length`

构造公式

二维前缀和:

```

1  pre[i][j] =
2      pre[i-1][j]
3      + pre[i][j-1]
4      - pre[i-1][j-1]
5      + matrix[i-1][j-1]

```

图形理解:

```

1      j
2
3  i  ┌   A   ┐
4      │       │
5      │       │
6      └       ┘

```

计算 `pre[i][j]` 时: 上方 + 左方 - 重复区域 + 当前元素

因为: `pre[i-1][j]` 和 `pre[i][j-1]` 重复算了 `pre[i-1][j-1]`, 所以要减一次。

O(1) 查询公式

查询: (r1,c1) 到 (r2,c2)

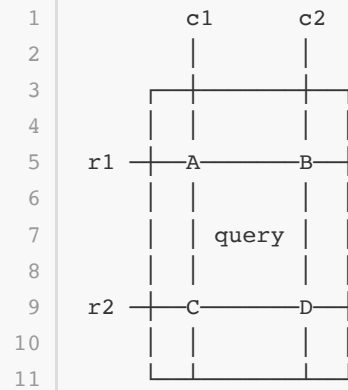
公式:


```

1 sum =
2     pre[r2+1][c2+1]
3     - pre[r1][c2+1]
4     - pre[r2+1][c1]
5     + pre[r1][c1]

```

图示：



计算：D - B - C + A

```

1 from typing import List
2
3 class NumMatrix:
4
5     def __init__(self, matrix: List[List[int]]):
6
7         n = len(matrix)
8         m = len(matrix[0])
9
10        self.pre = [[0]*(m+1) for _ in range(n+1)]
11
12        for i in range(1,n+1):
13            for j in range(1,m+1):
14                self.pre[i][j] = (
15                    self.pre[i-1][j]
16                    + self.pre[i][j-1]
17                    - self.pre[i-1][j-1]
18                    + matrix[i-1][j-1]
19                )
20
21        def sumRegion(self, r1: int, c1: int, r2: int, c2: int) -> int:
22
23            return (
24                self.pre[r2+1][c2+1]
25                - self.pre[r1][c2+1]
26                - self.pre[r2+1][c1]
27                + self.pre[r1][c1]
28            )
29

```

```
30
31 # Your NumMatrix object will be instantiated and called as such:
32 # obj = NumMatrix(matrix)
33 # param_1 = obj.sumRegion(row1,col1,row2,col2)
```

3 动态前缀和：树状数组（Binary Indexed Tree, BIT）

- 类别：数据结构
- 说明：树状数组是一种用于高效处理前缀和查询与单点更新的数据结构，支持 $O(\log n)$ 的更新和查询。虽然名字中有“树”，但它通常用数组实现，结构隐含在索引的二进制表示中。
- 典型应用：动态前缀和、逆序对计数、区间更新+单点查询（配合差分）等。

M307.区域和检索 - 数组可修改？

binary indexed tree, segment tree, <https://leetcode.cn/problems/range-sum-query-mutable/>

给你一个数组 `nums`，请你完成两类查询。

1. 其中一类查询要求 **更新** 数组 `nums` 下标对应的值
2. 另一类查询要求返回数组 `nums` 中索引 `left` 和索引 `right` 之间（包含）的 `nums` 元素的和，其中 `left <= right`

实现 `NumArray` 类：

- `NumArray(int[] nums)` 用整数数组 `nums` 初始化对象
- `void update(int index, int val)` 将 `nums[index]` 的值更新为 `val`
- `int sumRange(int left, int right)` 返回数组 `nums` 中索引 `left` 和索引 `right` 之间（包含）的 `nums` 元素的和（即，`nums[left] + nums[left + 1], ..., nums[right]`）

示例 1：

```
1  输入：
2  ["NumArray", "sumRange", "update", "sumRange"]
3  [[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
4  输出：
5  [null, 9, null, 8]
6
7  解释：
8  NumArray numArray = new NumArray([1, 3, 5]);
9  numArray.sumRange(0, 2); // 返回 1 + 3 + 5 = 9
10 numArray.update(1, 2);   // nums = [1,2,5]
11 numArray.sumRange(0, 2); // 返回 1 + 2 + 5 = 8
```

提示：

- `1 <= nums.length <= 3 * 10^4`
- `-100 <= nums[i] <= 100`
- `0 <= index < nums.length`
- `-100 <= val <= 100`
- `0 <= left <= right < nums.length`
- 调用 `update` 和 `sumRange` 方法次数不大于 `3 * 10^4`

```
1 class NumArray:
2
3     def __init__(self, nums: List[int]):
4
5
6     def update(self, index: int, val: int) -> None:
7
8
9     def sumRange(self, left: int, right: int) -> int:
10
11
12
13 # Your NumArray object will be instantiated and called as such:
14 # obj = NumArray(nums)
15 # obj.update(index, val)
16 # param_2 = obj.sumRange(left, right)
```

灵神讲的很清楚，证明我没看。里面还有视频讲解的链接。

带你发明树状数组！附数学证明

<https://leetcode.cn/problems/range-sum-query-mutable/solutions/2524481/dai-ni-fa-ming-s-hu-zhuang-shu-zu-fu-shu-lyfil/>

把一个正整数拆分，按照2的幂，从右往左拆分。拆分出的关键区间个数，是二进制数中1的个数是。位运算技巧。

二、如何拆分？

启示：如果把一个正整数 i 拆分成若干个不同的 2 的幂（从大到小），那么只会拆分成 $\mathcal{O}(\log i)$ 个数。前缀能否也这样拆分呢？

举个例子， $13 = 8 + 4 + 1$ ，那么前缀 $[1, 13]$ 可以拆分成三个长度分别为 8, 4, 1 的关键区间： $[1, 8], [9, 12], [13, 13]$ 。

按照这个规则，来看看从 $[1, 1]$ 到 $[1, 8]$ 是如何拆分的：

$[1, 1] = [1, 1]$	$(1 = 1)$
$[1, 2] = [1, 2]$	$(2 = 2)$
$[1, 3] = [1, 2] + [3, 3]$	$(3 = 2 + 1)$
$[1, 4] = [1, 4]$	$(4 = 4)$
$[1, 5] = [1, 4] + [5, 5]$	$(5 = 4 + 1)$
$[1, 6] = [1, 4] + [5, 6]$	$(6 = 4 + 2)$
$[1, 7] = [1, 4] + [5, 6] + [7, 7]$	$(7 = 4 + 2 + 1)$
$[1, 8] = [1, 8]$	$(8 = 8)$

数一数。按照这种拆分方式，总共有多少个不同的关键区间？（提示：把注意力放在区间的右端点上。）

有 8 个: $[1, 1], [1, 2], [3, 3], [1, 4], [5, 5], [5, 6], [7, 7], [1, 8]$ 。

一般地:

- 如果 i 是 2 的幂, 那么 $[1, i]$ 无需拆分。
- 如果 i 不是 2 的幂, 那么先拆分出一个最小的 2 的幂, 记作 $\text{lowbit}(i)$ (例如 6 拆分出 2), 得到长为 $\text{lowbit}(i)$ 的关键区间 $[i - \text{lowbit}(i) + 1, i]$, 问题转换成剩下的 $[1, i - \text{lowbit}(i)]$ 如何拆分, 这是一个规模更小的子问题。

总共有 n 个不同的关键区间。

证明: 按顺序拆分前缀 $[1, 1], [1, 2], [1, 3], \dots, [1, n]$, 每次只会恰好拆出一个新的关键区间 $[i - \text{lowbit}(i) + 1, i]$ (注意 $[1, i - \text{lowbit}(i)]$ 之前拆分过了, 不会产生新的关键区间), 所以一共有 n 个不同的关键区间。

算法

由于关键区间的右端点互不相同, 我们可以把右端点为 i 的关键区间的元素和保存在 $\text{tree}[i]$ 中。

按照如下方法计算前缀 $[1, i]$ 的元素和:

1. 初始化元素和 $s = 0$ 。
2. 每次循环, 把 $\text{tree}[i]$ 加到 s 中, 对应关键区间 $[i - \text{lowbit}(i) + 1, i]$ 的元素和。
3. 然后更新 i 为 $i - \text{lowbit}(i)$, 表示接下来要拆分 $[1, i - \text{lowbit}(i)]$, 获取其中关键区间的元素和。
4. 循环直到 $i = 0$ 为止。
5. 返回 s 。

由于正整数 i 的二进制长度是 $\lceil \log_2 i \rceil + 1$, 所以任意前缀至多拆分出 $\mathcal{O}(\log n)$ 个关键区间, 所以上述算法的时间复杂度为 $\mathcal{O}(\log n)$ 。

关于 $\text{lowbit}(i)$ 的计算方法, 请看 [从集合论到位运算, 常见位运算技巧分类总结!](#)

要计算 $\text{sumRange}(\text{left}, \text{right})$, 可以分别计算 $[1, \text{right} + 1]$ 的元素和 (改成下标从 1 开始), 以及 $[1, \text{left}]$ 的元素和, 两者相减即为答案。

接下来, 看看更新一个数组元素时, 会影响到哪些关键区间。

三、如何更新?

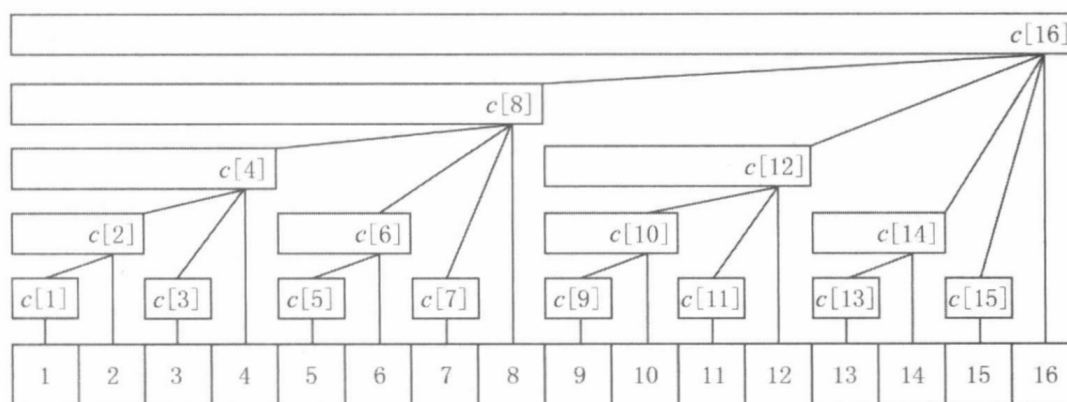
更新操作证明比较复杂, 推荐结合 [视频](#) 理解。

假设下标 x 发生了更新, 那么所有包含 x 的关键区间都会被更新。

例如下标 5 更新了, 那么关键区间 $[5, 5], [5, 6], [1, 8], [1, 16]$ 都需要更新, 这三个关键区间的右端点依次为 5, 6, 8, 16。

如果在 5-6, 6-8, 8-16 之间连边 (其它位置也同理), 我们可以得到一个什么样的结构?

如下图, 这些关键区间可以形成如下树形结构 (区间元素和保存在区间右端点处)。



注意到:

$$\begin{aligned} 5 + \text{lowbit}(5) &= 5 + 1 = 6 \\ 6 + \text{lowbit}(6) &= 6 + 2 = 8 \\ 8 + \text{lowbit}(8) &= 8 + 8 = 16 \end{aligned}$$

关于树状数组，重在理解其原理与应用，掌握其核心思想即可，无需深究形式化证明。这一数据结构设计精妙，其中对位运算的巧妙运用堪称点睛之笔，充分体现了算法的优雅与高效。值得一提的是，在利用 lowbit 实现前缀和查询时，常见的写法 $i -= i \& -i$ 可以等价地改写为 $i \&= i - 1$ 。两者语义完全相同，但后者更优——不仅代码更简洁，还少了一次算术运算，效率略高。

它的核心思想是将数组按特定规则分组进行高效检索：利用整数的二进制表示，将其按 2 的幂次进行划分，从而实现了对前缀信息的快速维护与查询。这一设计仅需 $O(\log n)$ 的时间复杂度，构思巧妙，堪称天才之作。

这个问题要求实现一个支持“单点修改”和“区域检索”的数据结构。

对于此类问题，普通的数组实现中：

- 数组/前缀和：update 是 $O(n)$ ，sumRange 是 $O(1)$ 。
- 普通数组：update 是 $O(1)$ ，sumRange 是 $O(n)$ 。

当两者调用次数都很多时（本题均为 3×10^4 ），需要更高效的数据结构。最常用的两种方案是：树状数组 (Binary Indexed Tree / Fenwick Tree) 和 线段树 (Segment Tree)。

树状数组 (Binary Indexed Tree)

树状数组是处理“动态前缀和”最简洁的工具。其核心思想是利用二进制低位的 lowbit 来管理不同长度的区间和。

- 时间复杂度：
 - 初始化： $O(n)$ 或 $O(n \log n)$
 - update： $O(\log n)$
 - sumRange： $O(\log n)$
- 空间复杂度： $O(n)$

```
1 class NumArray:
2     def __init__(self, nums: List[int]):
3         self.nums = nums
4         self.n = len(nums)
5         # tree 数组下标从 1 开始，所以长度为 n + 1
6         self.tree = [0] * (self.n + 1)
7         # 初始化树状数组
8         for i, val in enumerate(nums):
9             self._add(i + 1, val)
10
11     def _lowbit(self, x: int) -> int:
12         return x & -x
13
14     def _add(self, index: int, delta: int):
15         """在树状数组的 index 位置增加 delta"""
16         while index <= self.n:
17             self.tree[index] += delta
18             index += self._lowbit(index)
19
20     def _query(self, index: int) -> int:
21         """查询前缀和 [0...index]"""
```

```

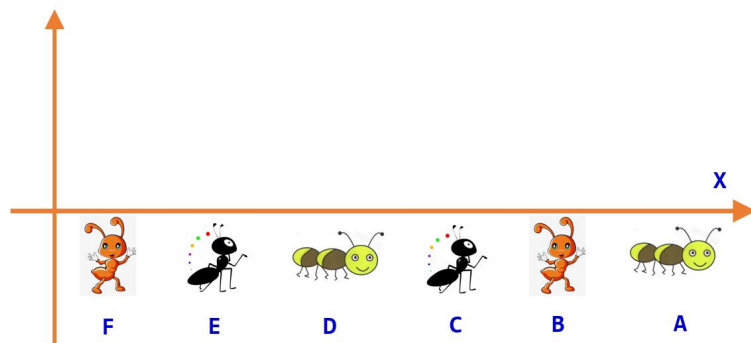
22         res = 0
23         while index > 0:
24             res += self.tree[index]
25             index -= self._lowbit(index)
26         return res
27
28     def update(self, index: int, val: int) -> None:
29         # 计算增量 delta
30         delta = val - self.nums[index]
31         self.nums[index] = val
32         # 树状数组内部是 1-indexed
33         self._add(index + 1, delta)
34
35     def sumRange(self, left: int, right: int) -> int:
36         # sum(left, right) = query(right) - query(left - 1)
37         return self._query(right + 1) - self._query(left)

```

M20018:蚂蚁王国的越野跑

merge sort, binary indexed tree, binary search, <http://cs101.openjudge.cn/practice/20018/>

为了促进蚂蚁家族身体健康，提高蚁族健身意识，蚂蚁王国举行了越野跑。假设越野跑共有N个蚂蚁参加，在一条笔直的道路上进行。N个蚂蚁在起点处站成一列，相邻两个蚂蚁之间保持一定的间距。比赛开始后，N个蚂蚁同时沿着道路向相同的方向跑去。换句话说，这N个蚂蚁可以看作x轴上的N个点，在比赛开始后，它们同时向X轴正方向移动。假设越野跑的距离足够远，这N个蚂蚁的速度有的不相同有的相同且保持匀速运动，那么会有多少对参赛者之间发生“赶超”的事件呢？此题结果比较大，需要定义long long类型。请看备注。



输入

第一行1个整数N。

第2... N + 1行：N 个非负整数，按从前到后的顺序给出每个蚂蚁的跑步速度。对于50%的数据， $2 \leq N \leq 1000$ 。对于100%的数据， $2 \leq N \leq 100000$ 。

输出

一个整数，表示有多少对参赛者之间发生赶超事件。

样例输入

```
1 sample1 input:
```

```
2 5
3 1
4 5
5 10
6 7
7 6
8
9 sample2 input:
10 5
11 1
12 5
13 5
14 7
15 6
```

样例输出

```
1 sample1 output:
2 7
3
4 sample2 output:
5 8
```

提示

我们把这5个蚂蚁依次编号为A,B,C,D,E，假设速度分别为1,5,5,7,6。在跑步过程中：B,C,D,E均会超过A，因为他们的速度都比A快；D,E都会超过B,C，因为他们的速度都比B,C快；D,E之间不会发生赶超，因为速度快的起跑时就在前边；B,C之间不会发生赶超，因为速度一样，在前面的就一直在前面。

考虑归并排序的思想。

此题结果比较大，需要定义long long类型，其输出格式为printf("%lld",x);

long long，有符号 64位整数，所占8个字节(Byte)

-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

这题本质上是 **统计逆序对**。

关键观察

蚂蚁一开始按位置从前到后排序：

若前面的蚂蚁 i 速度 $v[i]$ 小于 后面的蚂蚁 j 速度 $v[j]$ ，那么 j 一定会追上 i 。

因此需要统计： $i < j$ 且 $v[i] < v[j]$ 的对数。

直觉解 (bisect)

复杂度： $O(N^2)$

```

1  from bisect import bisect_left
2  n=int(input())
3  v=[]
4  ans=0
5  for i in range(n):
6      p=int(input())
7      index=bisect_left(v,p)
8      v.insert(index,p)
9      ans+=index
10 print(ans)

```

bisect_left 的作用

```
1 | index = bisect_left(v, p)
```

含义：在有序数组 `v` 中找到 `p` 应该插入的位置

并且这个位置满足：

```

1 | v[0:index] < p
2 | v[index:] >= p

```

因此：

```
1 | index = 已有元素中 < p 的个数
```

这正好就是需要统计的数量。

分治解（归并统计）

而 归并排序统计逆序对通常统计的是： `i < j` 且 `v[i] > v[j]`

所以只要把速度 取负数，就能直接用逆序对模板。

时间复杂度： $O(N \log N)$ ，适合 $N \leq 100000$ 。

Python实现（归并排序统计）

```

1  import sys
2  sys.setrecursionlimit(1000000)
3
4  def merge_sort(arr):
5      n = len(arr)
6      if n <= 1:
7          return arr, 0
8
9      mid = n // 2
10     left, cnt1 = merge_sort(arr[:mid])

```



```

11     right, cnt2 = merge_sort(arr[mid:])
12
13     i = j = 0
14     merged = []
15     cnt = cnt1 + cnt2
16
17     while i < len(left) and j < len(right):
18         if left[i] <= right[j]:
19             merged.append(left[i])
20             i += 1
21         else:
22             merged.append(right[j])
23             cnt += len(left) - i
24             j += 1
25
26     merged.extend(left[i:])
27     merged.extend(right[j:])
28
29     return merged, cnt
30
31
32 def main():
33     input = sys.stdin.readline
34     n = int(input())
35     v = [int(input()) for _ in range(n)]
36
37     # 转成负数, 把 v[i] < v[j] 转成逆序对
38     arr = [-x for x in v]
39
40     _, ans = merge_sort(arr)
41     print(ans)
42
43
44 if __name__ == "__main__":
45     main()

```

思维图（非常关键）

原序列：

```

1 | 位置： 1 2 3 4 5
2 | 速度： 1 5 10 7 6

```

追赶发生条件：

```

1 | 前面慢，后面快
2 | v[i] < v[j]

```

例如：

```
1 1 < 5
2 1 < 10
3 1 < 7
4 1 < 6
5 ...
```

统计这种对数即可。

为什么题目提示归并排序

因为：

- N 最大 **100000**
- 暴力 $O(N^2)$ 会 **超时**
- 归并统计逆序对是经典 $O(N \log N)$

数据结构解（BIT）

下面给出 **BIT**（树状数组）版本。这个题其实非常适合讲 **树状数组统计顺序对**。

我们要统计： $i < j$ 且 $v[i] < v[j]$

思路是 **从左到右扫描**：

- 已经扫描过的蚂蚁速度放进树状数组
- 当前蚂蚁速度为 v
- 查询 **之前有多少速度 $< v$**

这样就得到所有 (i, j) 。但速度范围可能很大，所以需要 **离散化**。

Python（BIT实现）

```
1 import sys
2 input = sys.stdin.readline
3
4 class BIT:
5     def __init__(self, n):
6         self.n = n
7         self.tree = [0]*(n+1)
8
9     def update(self, i, v=1):
10         while i <= self.n:
11             self.tree[i] += v
12             i += i & -i
13
14     def query(self, i):
15         s = 0
16         while i > 0:
17             s += self.tree[i]
```

```

18         i -= i & -i
19         return s
20
21
22 n = int(input())
23 v = [int(input()) for _ in range(n)]
24
25 # 1. 离散化
26 vals = sorted(set(v))
27 rank = {x:i+1 for i,x in enumerate(vals)}
28
29 # 2. BIT
30 bit = BIT(len(vals))
31
32 ans = 0
33
34 # 3. 从左到右扫描
35 for x in v:
36     r = rank[x]
37
38     # 统计之前速度 < x
39     ans += bit.query(r-1)
40
41     # 当前速度加入
42     bit.update(r)
43
44 print(ans)

```

运行逻辑示例

输入：

```
1 | 1 5 10 7 6
```

扫描过程：

当前蚂蚁	速度	之前更慢的数量	累计
1	1	0	0
2	5	1	1
3	10	2	3
4	7	2	5
5	6	2	7

结果：

```
1 | 7
```

复杂度

- 离散化: $O(N \log N)$
- BIT操作: $O(N \log N)$

总体: $O(N \log N)$, 适合 $N=10^5$ 。

课堂讲解建议

这题特别适合讲三种方法的对比:

- 1 暴力 $O(N^2)$
- 2 归并排序统计逆序对 $O(N \log N)$
- 3 树状数组统计顺序对 $O(N \log N)$

4 归并排序 (Merge Sort)

- 类别: 算法 (排序算法 / 分治算法)
- 说明: 归并排序是一种经典的分治算法, 时间复杂度稳定为 $O(n \log n)$, 可用于排序, 也可用于解决如逆序对计数等问题。
- 典型应用: 排序、求逆序对、外部排序等。

M02299: Ultra-QuickSort

merge sort, <http://cs101.openjudge.cn/practice/02299/>

In this problem, you have to analyze a particular sorting algorithm. The algorithm processes a sequence of n distinct integers by swapping two adjacent sequence elements until the sequence is sorted in ascending order. For the input sequence 9 1 0 5 4, Ultra-QuickSort produces the output 0 1 4 5 9. Your task is to determine how many swap operations Ultra-QuickSort needs to perform in order to sort a given input sequence.

输入

The input contains several test cases. Every test case begins with a line that contains a single integer $n < 500,000$ -- the length of the input sequence. Each of the the following n lines contains a single integer $0 \leq a[i] \leq 999,999,999$, the i -th input sequence element. Input is terminated by a sequence of length $n = 0$. This sequence must not be processed.

输出

For every input sequence, your program prints a single line containing an integer number op , the minimum number of swap operations necessary to sort the given input sequence.

样例输入

1	5
2	9
3	1
4	0
5	5
6	4
7	3
8	1
9	2
10	3
11	0

样例输出

1	6
2	0

来源

Waterloo local 2005.02.05

M20018:蚂蚁王国的越野跑

merge sort, binary indexed tree, binary search, <http://cs101.openjudge.cn/practice/20018/>

代码放在上面 树状数组 中，即三种解法之一的归并排序求逆序数。

附录：

A.树状数组的额外说明

树状数组或二叉索引树（英语：Binary Indexed Tree），又以其发明者命名为Fenwick树，最早由 Peter M. Fenwick于1994年以A New Data Structure for Cumulative Frequency Tables为题发表。其初衷是解决数据压缩里的累积频率（Cumulative Frequency）的计算问题，现多用于高效计算数列的**前缀和**，**区间和**。

一般来说，如果在查询的过程中元素可能发生改变（例如插入、修改或删除），就称这种查询为**在线查询**；如果在查询过程中元素不发生改变，就称为**离线查询**。

二叉索引树（树状数组）用于处理对固定大小的数组进行以下多种操作的这类问题。

- 前缀操作（求和、求积、异或、按位或等）。注意，区间操作也可以通过前缀来解决。例如，从索引L到R的区间和等于到R（包含R）的前缀和减去到L - 1的前缀和。
- 更新数组中的一个元素

这两种操作的时间复杂度均为 $O(\log N)$ 。注意，我们需要 $O(N \log N)$ 的预处理时间和 $O(N)$ 的辅助空间。

让我们考虑以下问题来理解二叉索引树 (Binary Indexed Tree, BIT) :

我们有一个数组 $arr[0 \dots n - 1]$ 。我们希望实现两个操作:

1. 计算前 i 个元素的和。
2. 修改数组中指定位置的值, 即设置 $arr[i] = x$, 其中 $0 \leq i \leq n - 1$ 。

一个简单的解决方案是从 0 到 $i-1$ 遍历并计算这些元素的总和。要更新一个值, 只需执行 $arr[i] = x$ 。第一个操作的时间复杂度为 $O(N)$, 而第二个操作的时间复杂度为 $O(1)$ 。另一种简单的解决方案是创建一个额外的数组, 并在这个新数组的第 i 个位置存储前 i 个元素的总和。这样, 给定范围的和可以在 $O(1)$ 时间内计算出来, 但是更新操作现在需要 $O(N)$ 时间。当查询操作非常多而更新操作非常少时, 这种方法表现良好。

我们能否在 $O(\log N)$ 时间内同时完成查询和更新操作呢?

一种高效的解决方案是使用段树 (Segment Tree), 它能够在 $O(\log N)$ 时间内完成这两个操作。

另一种解决方案是二叉索引树 (Binary Indexed Tree, 也称作 Fenwick Tree), 同样能够以 $O(\log N)$ 的时间复杂度完成查询和更新操作。与段树相比, 二叉索引树所需的空间更少, 且实现起来更加简单。

lowbit 运算

二进制中一个经典应用是 lowbit 运算, 即 $\text{lowbit}(x) = x \& (-x)$ 。

整数的二进制表示常用的方式之一是使用补码

补码是一种表示有符号整数的方法, 它将负数的二进制表示转换为正数的二进制表示。补码的优势在于可以使用相同的算术运算规则来处理正数和负数, 而不需要特殊的操作。

在补码表示中, 最高位用于表示符号位, 0 表示正数, 1 表示负数。其他位表示数值部分。

具体将一个整数转换为补码的步骤如下:

1. 如果整数是正数, 则补码等于二进制表示本身。
2. 如果整数是负数, 则需要先将其绝对值转换为二进制, 然后取反, 最后加1。等价于把二进制最右边的1的左边的每一位都取反。

例如, 假设要将 -12 转换为补码:

1. 12 的二进制表示为 00001100。
2. 将其取反得到 11110011。
3. 加1得到 11110100, 这就是 -12 的补码表示。

通过 $\text{lowbit}(x) = x \& (-x)$ 就是取 x 的二进制最右边的1和它右边所有的0, 因此它一定是2的幂次, 即1、2、4、8等。

对 $x = 12 = (00001100)_2$, 有 $-x = (11110100)_2$, $x \& (-x) = 4$

对 $x = 6 = (110)_2$, 有 $-x = (010)_2$, $x \& (-x) = 2$

表示方式

树状数组 (Binary Indexed Tree, BIT) 用数组形式表示。它其实仍然是一个数组, 并且与 sum 数组类似, 是一个用来记录和的数组, 只不过它存放的不是前 i 个整数之和, 而是在 i 号位之前 (含 i 号位) $\text{lowbit}(i)$ 个整数之和。树状数组的大小等于输入数组的大小, 记为 n 。在下面的代码中, 为了便于实现, 使用 $n+1$ 的大小。

如下图所示, 数组 A 是原始数组, 有 $A[1] \sim A[16]$ 共 16 个元素; 数组 C 是树状数组, 其中 $C[i]$ 存放数组 A 中 i 号位之前 $\text{lowbit}(i)$ 个元素之和。显然, $C[i]$ 的覆盖长度是 $\text{lowbit}(i)$ (也可以理解成管辖范围), 它是 2 的幂次, 即 1、2、4、8 等。

需要注意的是, 树状数组仍旧是一个平坦的数组, 画成树形是为了让存储的元素更容易观察。

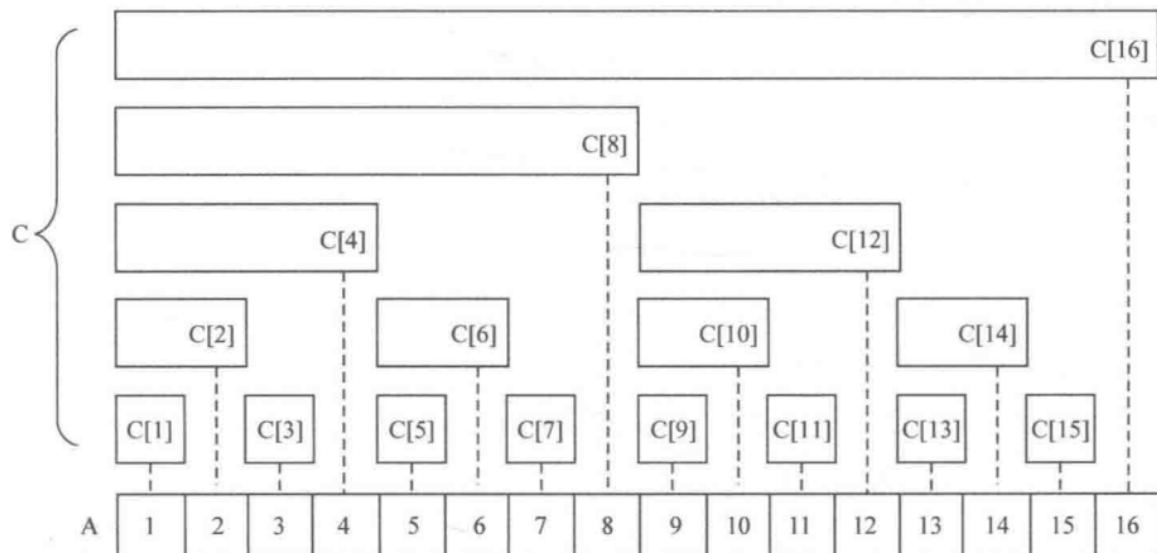


图 树状数组定义图

```

1  C[1] = A[1]                      (长度为 lowbit(1) = 1)
2  C[2] = A[1] + A[2]              (长度为 lowbit(2) = 2)
3  C[3] = A[3]                      (长度为 lowbit(3) = 1)
4  C[4] = A[1] + A[2] + A[3] + A[4] (长度为 lowbit(4) = 4)
5  C[5] = A[5]                      (长度为 lowbit(5) = 1)
6  C[6] = A[5] + A[6]              (长度为 lowbit(6) = 2)
7  C[7] = A[7]                      (长度为 lowbit(7) = 1)
8  C[8] = A[1] + A[2] + A[3] + A[4] + A[5] + A[6] + A[7] + A[8] (长度为
    lowbit(8) = 8)

```

树状数组的定义非常重要, 特别是“ $C[i]$ 的覆盖长度是 $\text{lowbit}(i)$ ”这点; 另外, 树状数组的下标必须从 1 开始。接下来思考一下, 在这样的定义下, 怎样解决下面两个问题:

- ① 设计函数 `get_sum(x)`, 返回前 x 个数之和 $A[1] + \dots + A[x]$ 。
- ② 设计函数 `update_bit(x,v)`, 实现将第 x 个数加上一个数 v 的功能, 即 $A[x] += v$ 。

先来看第一个问题, 即如何设计函数 `get_sum(x)`, 返回前 x 个数之和。不妨先看个例子。假设想要查询 $A[1] + \dots + A[14]$, 那么从树状数组的定义出发, 它实际是什么东西呢? 回到上图, 很容易发现 $A[1] + \dots + A[14] = C[8] + C[12] + C[14]$ 。又比如要查询 $A[1] + \dots + A[11]$, 从图中同样可以得到 $A[1] + \dots + A[11] = C[8] + C[10] + C[11]$ 。那么怎样知道 $A[1] + \dots + A[x]$ 对应的是树状数组中的哪些项呢? 事实上这很简单。记 $\text{SUM}(1,x) = A[1] + \dots + A[x]$, 由于 $C[x]$ 的覆盖长度是 $\text{lowbit}(x)$, 因此

$$C[x] = A[x - \text{lowbit}(x) + 1] + \dots + A[x]$$

于是可以得到

```
1 SUM(1,x) = A[1] + ... + A[x]
2           = A[1] + ... + A[x - lowbit(x)] + A[x - lowbit(x) + 1] + ... + A[x]
3           = SUM(1, x - lowbit(x)) + C[x]
```

这样就把 SUM(1,x)转换为 SUM(1,x-lowbit(x))了。

接着就能写出 get_sum 函数了，其中BITTree是树状数组。

```
1 def bit_sum(BIT, i):
2     s = 0
3     i += 1 # index in BIT[] is 1 more than the index in arr[]
4
5     while i > 0: # Traverse ancestors of BIT[index]
6         s += BIT[i]
7         i -= i & (-i) # Move index to parent node
8     return s
```

由于 lowbit(i)的作用是定位i的二进制中最右边的1，因此 `i = i - lowbit(i)` 事实上是不断把i的二进制中最右边的1置为0的过程。所以 get_sum 函数的 for 循环执行次数为x的二进制中1的个数。一个数n的二进制表示中设置位的数量是 $O(\log n)$ 。也就是说，get_sum 函数的时间复杂度为 $O(\log N)$ 。从另一个角度理解，结合图会发现，get_sum 函数的过程实际上是在沿着一条不断左上的路径行进（可以想一想 get_sum(14)跟 get_sum(11)的过程）。于是由于“树”高是 $O(\log N)$ 级别，因此可以同样得到 get_sum 函数的时间复杂度就是 $O(\log N)$ 。另外，如果要求数组下标在区间[x,y]内的数之和，即 $A[x] + A[x+1] + \dots + A[y]$ ，可以转换成 $\text{get_sum}(y) - \text{get_sum}(x-1)$ 来解决，这是一个很重要的技巧。

接着来看第二个问题，即如何设计函数 update(x,v)，实现将第x个数加上一个数v的功能。

来看两个例子。假如要让 A[6]加上一个数 v，那么就要寻找树状数组C中能覆盖了 A[6]的元素，让它们都加上 v。也就是说，如果要想 A[6]加上 v，实际上是要让 C[6]、C[8]、C[16]都加上 v。同样，如果要将 A[9]加上一个数 v，实际上就是要让 C[9]、C[10]、C[12]、C[16]都加上 v。于是问题又来了——想要给 A[x]加上v时，怎样去寻找树状数组中的对应项呢？

要让 A[x]加上 v，就是要寻找树状数组 C 中能覆盖 A[x]的那些元素，让它们都加上 v。而从图 1 中直观地看，只需要总是寻找离当前的“矩形”C[x]最近的“矩形”C[y]，使得 C[y]能够覆盖 C[x]即可。例如要让 A[6]加上 v，就从 C[6]开始找起：离 C[6]最近的能覆盖 C[6]的“矩形”是 C[8]，离 C[8]最近的能覆盖 C[8]的“矩形”是 C[16]，于是只要把 C[6]、C[8]、C[16]都加上v即可。

那么，如何找到距离当前的 C[x]最近的能覆盖 C[x]的 C[y]呢？首先，可以得到一个显然的结论：lowbit(y)必须大于 lowbit(x)（不然怎么覆盖呢.....）。于是问题等价于求一个尽可能小的整数 a，使得 $\text{lowbit}(x+a) > \text{lowbit}(x)$ 。显然，由于 lowbit(x)是取x的二进制最右边的1的位置，因此如果 $\text{lowbit}(a) < \text{lowbit}(x)$ ， $\text{lowbit}(x+a)$ 就会小于 lowbit(x)。为此 lowbit(a)必须不小于 lowbit(x)。接着发现，当a取 lowbit(x)时，由于x和a的二进制最右边的1的位置相同，因此x+a会在这个1的位置上产生进位，使得进位过程中的所有连续的1变成0，直到把它们左边第一个0置为1时结束。于是 $\text{lowbit}(x+a) > \text{lowbit}(x)$ 显然成立，最小的a就是lowbit(x)。于是 update 函数的做法就很明确了，只要让x不断加上 lowbit(x)，并让每步的 C[x]都加上 v，直到x超过给定的数据范围为止。代码如下：


```

1 def bit_update(BIT, n, i, v):
2     i += 1 # index in BITree[] is 1 more than the index in arr[]
3
4     while i <= n: # Traverse all ancestors and add 'val'
5         BIT[i] += v
6         i += i & (-i) # Update index to that of parent

```

更新函数需要确保所有包含arr[i]在其范围内的BIT节点都被更新。我们通过不断向当前索引添加其最后一位设置位对应的十进制数，在BIT中循环遍历这些节点。

实现

首先将BIT[]中的所有值初始化为0。然后对所有的索引调用bit_update()函数。

```

1 # Binary Indexed Tree
2
3 def bit_sum(BIT, i):
4     """计算树状数组 BIT 从索引 1 到 i 的前缀和"""
5     s = 0
6     while i > 0:
7         s += BIT[i]
8         i -= i & (-i) # 回溯至祖先节点
9     return s
10
11
12 def bit_update(BIT, i, v):
13     """在树状数组 BIT 中更新索引 i 处的值 v"""
14     while i < len(BIT):
15         BIT[i] += v
16         i += i & (-i) # 回溯至祖先节点
17
18
19 # Constructs and returns a Binary Indexed Tree for given array of size n.
20 def construct(arr, n):
21     BIT = [0] * (n + 1)
22     for i in range(n): # Store the actual values in BIT[] using
23         bit_update(BIT, i + 1, arr[i])
24
25     return BIT
26
27
28 arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16]
29 BIT = construct(arr, len(arr))
30 print(f'BIT: ', *BIT)
31 print("Sum of elements in arr[0..5] is " + str(bit_sum(BIT, 5)))
32 arr[3] += 6
33 bit_update(BIT, 3, 6)
34 print(f'BIT: ', *BIT)
35 print("Sum of elements in arr[0..5]" +
36       " after update is " + str(bit_sum(BIT, 5)))
37

```

Output

```
1 BIT:  0 1 3 3 10 5 11 7 36 9 19 11 42 13 27 15 136
2 Sum of elements in arr[0..5] is 15
3 BIT:  0 1 3 9 16 5 11 7 42 9 19 11 42 13 27 15 142
4 Sum of elements in arr[0..5] after update is 21
```

Time Complexity: $O(N \log N)$

Auxiliary Space: $O(N)$

Can we extend the Binary Indexed Tree to computing the sum of a range in $O(\log n)$ time?

Yes. $\text{rangeSum}(l, r) = \text{get_sum}(r) - \text{get_sum}(l-1)$.

References:

http://en.wikipedia.org/wiki/Fenwick_tree